



45. Transfer Learning

Leverage the Past to Power the Future

Previous Section:

 [44. Leveraging Pre-Trained Models](#)

Contents:

[1. Transfer Learning – The Overview](#)

[2. How Transfer Learning differs from Traditional Machine Learning](#)

[3. When to Freeze and Fine-Tune Layers in Transfer Learning](#)

[4. Applications of Transfer Learning](#)

[5. Transfer Learning with Python](#)

[Summary](#)

[References](#)

Every new concept in Machine Learning starts the same way: it sounds simpler than it really is.

“Transfer Learning?” → It almost feels obvious. Learn something here, use it there. And in a sense, that’s exactly what it is. But like most things in Machine Learning, the idea is simple. The execution? Not so much.

You’ve already seen how Machine Learning models learn from data. They observe patterns, understand relationships, and eventually make predictions. But there’s a hidden assumption behind all of that:

We usually train models from scratch: ***We collect data → train a model → let it learn everything on its own.***

And that works, until it doesn't. Because here's the problem: Learning from scratch is expensive. It takes huge amounts of data; significant computational power; to put it simply, a lot of time.

And sometimes, we simply don't have those resources. So now the real question becomes: What if the model didn't have to start from zero?

Think about how humans learn. If you already know how to ride a bicycle, learning to ride a motorcycle becomes easier. Not because they are identical, but because some knowledge carries over. Balance. Coordination. Awareness.

The idea: You don't relearn everything. You reuse what you already know.

Transfer Learning follows the exact same philosophy. Instead of training a model from scratch, we take a model that has already learned something useful, and adapt it to a new problem.

A model trained on millions of images already understands: Edges; Shapes; Patterns

So when we use it for a different task—like medical image analysis—we're not starting from nothing. We're building on top of existing knowledge. And that changes everything. Because now, we need less data. Training becomes faster. Performance often improves.

So in a way, Transfer Learning is not just a technique. It's a shift in mindset. Instead of asking: "How do we teach a model everything?" → We ask something smarter: *"What does the model already know... and how can we reuse it?"*

And in the next section, we'll turn this intuition into something more concrete.

1. Transfer Learning – The Overview

Now let's take that idea we just built, and turn it into something clearer. What exactly is Transfer Learning?

At its core, Transfer Learning is about reuse. Not reuse in a lazy way, but in a very intelligent way.

Instead of training a model from scratch every single time... we take a model that has already learned something useful, and use it as the starting point for a new problem.

So rather than asking: "How do we build a model from zero?" → We ask something better: *"What has already been learned... and how can we use it?"*

Think about what happens when we train a model normally. We give it data, a lot of data. It slowly learns simple patterns, then more complex structures, and eventually meaningful representations

But this process is expensive. It takes time. It takes data. It takes computational power. And sometimes, we simply don't have enough of those.

This is where Transfer Learning becomes powerful. Because instead of starting from nothing... we start from experience. A pretrained model, one that has already been trained on a large dataset, has already learned a wide range of patterns. Some of these patterns are very general: Edges in images; Basic shapes; Common structures. Others are more task-specific. Transfer Learning focuses on something important: *Not everything the model learned is useful, but a lot of it is reusable.*

So how does the process actually work? Let's walk through it step by step.

- **First, we begin with a pretrained model.**

This model has already been trained on a large dataset, often millions of examples. It already understands a lot about the structure of the data.

At this point, we don't treat it as a finished solution. We treat it as a starting point.

- **Next, this pretrained model becomes our base model.**

Instead of building a new model from scratch, we reuse this one. Because inside this model, there are layers. And each layer has learned something different. Early layers usually learn very general patterns. Deeper layers learn more specific and complex representations.

And here's the key idea: We don't need all of it. We only keep what is useful.

So we select the layers that capture general knowledge. These are the parts of the model that understand patterns that are not tied to just one specific task. And in many cases, we freeze them.

Freezing means something very simple: We do not update their parameters during training. Why? Because they already contain valuable knowledge. If we change them too much, especially with a small dataset, we risk destroying what they learned. And worse, we might overfit.

- **Finally, we fine-tune the model.**

This is where the model adapts to the new task. We allow some layers, usually the later ones, to learn from the new dataset. So now the model does two things at the same time: It keeps what it already knows, and it adjusts itself to what is new. That balance? That's where Transfer Learning works best.

If all of this still feels abstract, think about it like this. Imagine a student. They spend years learning Mathematics. Algebra. Calculus. Probability. Now they start learning Machine Learning. Do they start from zero? Of course not. They already understand equations; optimization; relationships between variables. So when they see concepts like regression or loss functions, it clicks faster. They are not relearning everything. They are transferring knowledge.

In this analogy:

- Mathematics → the pretrained model
- Fundamental concepts → general features
- Machine Learning → the new task
- And the learning process? → That's fine-tuning.

So in the end, Transfer Learning is not just about saving time. It's about learning smarter. Reuse what is already known, preserve what is useful, adapt only what is necessary. And let the model build on experience, instead of starting from scratch.

2. How Transfer Learning differs from Traditional Machine Learning

Now that we understand what Transfer Learning is, let's look at something just as important: What makes it different? Because at first glance... it might feel like just another technique. But in reality, it represents a completely different way of thinking about learning.

Let's start with Traditional Machine Learning. In the traditional approach, every problem is treated as independent. New dataset? New model. Another dataset? Another model.

And each time, the process repeats: Train from scratch. Learn patterns from zero. Build everything again.

There is no memory. No reuse. No connection between tasks. If you have 10 different datasets, you train 10 separate models. Each one lives in its own

world.

Now compare that with Transfer Learning. Transfer Learning does something much smarter. It assumes that learning is not isolated, it's cumulative. Instead of starting over every time, it asks:

■ ***"What have we already learned... and how can we use it?"***

So rather than building a completely new model, we begin with one that already has experience.

Let's break down how this actually works.

- First, a model is trained on a large dataset. This is called the **source task**. During this stage, the model learns a wide range of patterns:
 - Basic structures
 - General relationships
 - Useful representations

Think of this as building a strong foundation.

Next, we move to a new problem. This is called the **target task**. Now here's where the shift happens: We don't throw away the old model. We reuse it. The knowledge learned from the source task becomes the starting point for the new task. And instead of learning everything again, the model only needs to adapt.

Finally, we fine-tune. We adjust parts of the model using the new dataset so it can better fit the specific problem. So the process becomes: ***Learn once → Reuse → Adapt → Improve***. Not: ***Learn → Forget → Restart***

This difference might seem subtle, but it has huge consequences.

In Traditional Machine Learning:

- Training takes longer
- Large labeled datasets are required
- Computational cost increases with every new task
- Models remain isolated and specialized

In Transfer Learning:

- Training becomes faster

- Less data is needed
- Knowledge is reused instead of discarded
- Models become more flexible and scalable

Another way to see it, Traditional Machine Learning builds many independent experts. Each expert knows one thing, and only that thing. Transfer Learning builds experience. It creates systems that learn over time, and carry that knowledge forward.

And if this feels familiar, it should. Because this is exactly how humans learn. When you study a new subject, you don't start from zero. You rely on what you already know. Math helps with Machine Learning. Logic helps with programming. Experience builds on experience.

Transfer Learning brings that same idea into Machine Learning. So in the end, the difference is not just technical. It's philosophical.

Traditional Machine Learning says: "Every problem is new." → Transfer Learning says: "Every problem is connected."

3. When to Freeze and Fine-Tune Layers in Transfer Learning

Now we reach one of the most practical—and often confusing—questions in Transfer Learning: "***What do we freeze... and what do we fine-tune?***"

Because remember, not all parts of a pre-trained model are equal. Some layers learn very general patterns. Others learn very specific details. So the real decision becomes: **What should we keep? And what should we adapt?**

The answer depends on two things: How much data you have and how similar your new task is to the original one. That's it. Two variables. But they change everything.

Let's go through the four common scenarios.

- **Small dataset + Similar task:** This is the safest and most common case.

Your new data looks similar to what the model has already seen, but you don't have much of it. So what do we do?

We freeze most of the model. And only fine-tune the final layers. Why? Because the model already understands the general patterns. If we try to update too many parameters with limited data, we risk overfitting.

So instead, we make small adjustments. Think of it like this: "***The model already 'gets it'... we're just helping it adapt slightly.***"

- **Large dataset + Similar task:** Now we still have similarity, but we also have enough data. This changes things.

We can now afford to fine-tune more layers. Not just the final ones, but deeper parts of the model as well. Because the data is large enough to support learning. The similarity ensures we are not starting from scratch. So the model can refine its knowledge instead of just slightly adjusting it.

- **Small dataset + Different task:** This is where things get tricky. The dataset is small, but also very different. So the patterns the model learned before? They may not fully apply anymore.

In this case, we often fine-tune layers closer to the input. Why? Because even the low-level features may need to change. The model needs to relearn more fundamental patterns, but with limited data. So we carefully adjust more layers—while still being cautious about overfitting.

- **Large dataset + Different task:** This is the most flexible scenario. You have a lot of data and a completely different problem

So what do we do? We fine-tune almost everything. Sometimes... even the entire model. Because now we have enough data to support full adaptation. The pre-trained model is no longer just a feature extractor. It becomes a strong starting point that we reshape completely.

So if you want one simple rule to remember:

The more data you have → the more you can fine-tune
The more similar the task → the more you can freeze

Everything else is just a combination of these two ideas.

And this is what makes Transfer Learning so powerful. It's not rigid. It's flexible. It allows you to adjust how much you reuse, and how much you relearn.

4. Applications of Transfer Learning

So far..., everything we've discussed has been conceptual. Now let's answer a more practical question: **Where is Transfer Learning actually used?**

Short answer: Almost everywhere.

But let's break it down into the most important areas.

- **Computer Vision — “Learning to see”**

This is where Transfer Learning shines the most. Models trained on massive image datasets already understand edges, shapes, textures. So when we apply them to new tasks—like medical image analysis, face recognition, object detection. We don't start from zero. We reuse visual understanding, and adapt it. That's why even complex vision systems can be built with relatively small datasets.

- **Natural Language Processing (NLP) — “Learning to understand language”**

In NLP, Transfer Learning is not just useful. It's essential. Modern models are first trained on massive amounts of text. They learn grammar, context, word relationships, meaning. Models like BERT or GPT don't start as task-specific systems. They start as general language learners. Then we fine-tune them for sentiment analysis, translation, text classification, question and answering. So instead of teaching language from scratch, we adapt existing understanding.

- **Deep Learning — “Scaling intelligence”**

Deep neural networks are powerful, but expensive. They require huge datasets and massive computation.

Transfer Learning changes that. Instead of training deep models from zero... we reuse pre-trained networks. This allows us to reduce training time, lower computational cost, but still achieve strong performance. In many real-world systems, this is the default approach.

- **Healthcare — “Learning from limited data”**

In healthcare, data is valuable, and scarce. Medical images like X-rays, CT scans, MRI scans. Are difficult and expensive to label.

Transfer Learning helps bridge that gap. We take models trained on general images. And adapt them to detect diseases, abnormalities, or patterns in medical data. So even with limited data, we can build useful diagnostic systems.

- **Finance — “Detecting patterns and risks”**

Financial systems deal with fraud detection, credit scoring, risk analysis. But labeled data is often limited or sensitive.

Transfer Learning allows models to reuse knowledge from related datasets. So instead of learning everything from scratch, they focus on adapting patterns, detecting anomalies, recognizing suspicious behavior, identifying hidden risks

So what's the common thread across all these fields? Different domains.

Different data. Different problems. But the same idea: **Don't waste what has already been learned.** Reuse it. Adapt it. Build on it.

And that's why Transfer Learning has become one of the most important ideas in modern Machine Learning. Because in the real world, starting from zero is rarely the best option.

5. Transfer Learning with Python

Now let's put Transfer Learning into code. Unfortunately, Transfer Learning is related heavily to Neural Networks. But don't worry about it too much. The main focus here is not the complexity of the model—but the idea of reusing a pre-trained model and adapting it to a new task.

In our case, we are working with image classification using the MNIST dataset, but instead of building a model from scratch, we will reuse a powerful pre-trained model called **MobileNetV2**.

Step 1: Preparing the Data

Before anything else, we need to make MNIST compatible with MobileNetV2. Why? Because MobileNetV2 was trained on ImageNet, which expects:

- 3-channel RGB images (not grayscale)
- Larger image sizes than 28×28

So we adjust MNIST like this:

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import numpy as np

# Load the MNIST dataset
(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()

# Convert grayscale → RGB (3 channels)
train_images = np.stack([train_images]*3, axis=-1)
test_images = np.stack([test_images]*3, axis=-1)

# Normalize pixels
train_images = train_images / 255.0
test_images = test_images / 255.0

# Resize images
train_images = tf.image.resize(train_images, [32, 32])
test_images = tf.image.resize(test_images, [32, 32])

train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)

print("Training data shape:", train_images.shape, train_labels.shape)
print("Testing data shape:", test_images.shape, test_labels.shape)

```

Training data shape: (60000, 32, 32, 3) (60000, 10)

Testing data shape: (10000, 32, 32, 3) (10000, 10)

In simple terms: We are just reshaping MNIST so that a pre-trained model can “understand” it.

Step 2: Building the Transfer Learning Model

Now we load MobileNetV2. The key idea:

- We reuse the learned feature extractor
- We replace the final classification layer

```
from tensorflow.keras.applications import MobileNetV2

base_model = MobileNetV2(weights='imagenet', include_top=False,
                           input_shape=(32, 32, 3))

base_model.trainable = False # freeze pretrained knowledge
```

This means: We are locking the “knowledge” of the model so it doesn’t forget what it already learned from ImageNet.

Now we add our own classifier on top:

```
from tensorflow.keras.layers import Input, GlobalAveragePooling2D, Dense
from tensorflow.keras.models import Model

inputs = Input(shape=(32, 32, 3))

x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)

outputs = Dense(10, activation='softmax')(x)

model = Model(inputs, outputs)
```

Simple interpretation:

- Base model = feature extractor (edges, shapes, patterns)
- New Dense layer = MNIST digit classifier (0–9)

Then we compile and train the model:

```

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(train_images, train_labels,
          epochs=5, batch_size=64, validation_split=0.2)

```

Step 3: Fine-Tuning the Model

Now we make the model slightly smarter. Instead of keeping everything frozen, we:

- Unfreeze part of the model
- Allow it to slightly adjust to MNIST

```

base_model.trainable = True

# Freeze early layers (keep general features)
for layer in base_model.layers[:100]:
    layer.trainable = False

```

Idea:

- Early layers = edges, shapes → keep them
- Later layers = more task-specific → allow learning

Then we retrain with a smaller learning rate:

```

model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(train_images, train_labels,
          epochs=5, validation_split=0.2)

```

Why smaller learning rate? Because we don't want to "destroy" what the pre-trained model already learned.

Step 4: Evaluation

Now we test the model:

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)
```

Test Loss: 0.39963072538375854

Test Accuracy: 0.8810999989509583

Meaning: The model is already performing well, even though it was not trained from scratch.

Step 5: Checking Predictions

```
y_pred = model.predict(test_images)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(test_labels, axis=1)
```

Then we compare predictions with true labels using:

- Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_true_classes, y_pred_classes))
```

```
precision  recall f1-score  support
```

```
0    0.94    0.97    0.96    980
1    0.97    0.92    0.94   1135
2    0.77    0.89    0.82   1032
3    0.85    0.86    0.85   1010
4    0.83    0.91    0.87    982
5    0.85    0.78    0.81    892
6    0.94    0.93    0.94    958
7    0.88    0.83    0.86   1028
8    0.90    0.85    0.87    974
9    0.90    0.87    0.89   1009
```

```
accuracy                0.88   10000
macro avg    0.88    0.88    0.88   10000
weighted avg    0.88    0.88    0.88   10000
```

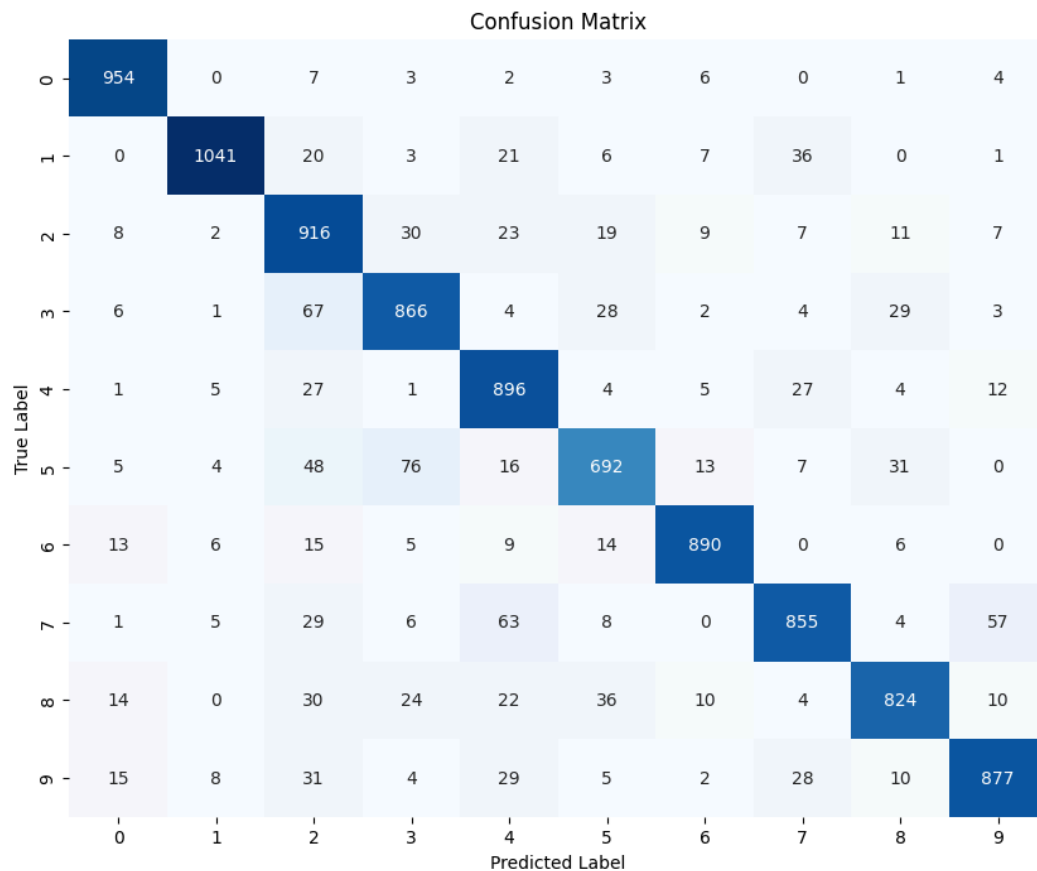
- Confusion Matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

test_predictions = model.predict(test_images)
test_predictions_classes = np.argmax(test_predictions, axis=1)
test_true_classes = np.argmax(test_labels, axis=1)
cm = confusion_matrix(test_true_classes, test_predictions_classes)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.xlabel('Predicted Label')
```

```
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```



This helps us see which digits are confused and where the model makes mistakes

Step 6: Visualizing Sample Predictions

```
import matplotlib.pyplot as plt

def display_sample(sample_images, sample_labels, sample_predictions):
    fig, axes = plt.subplots(3, 3, figsize=(12, 12))
    fig.subplots_adjust(hspace=0.5, wspace=0.5)
    for i, ax in enumerate(axes.flat):
```

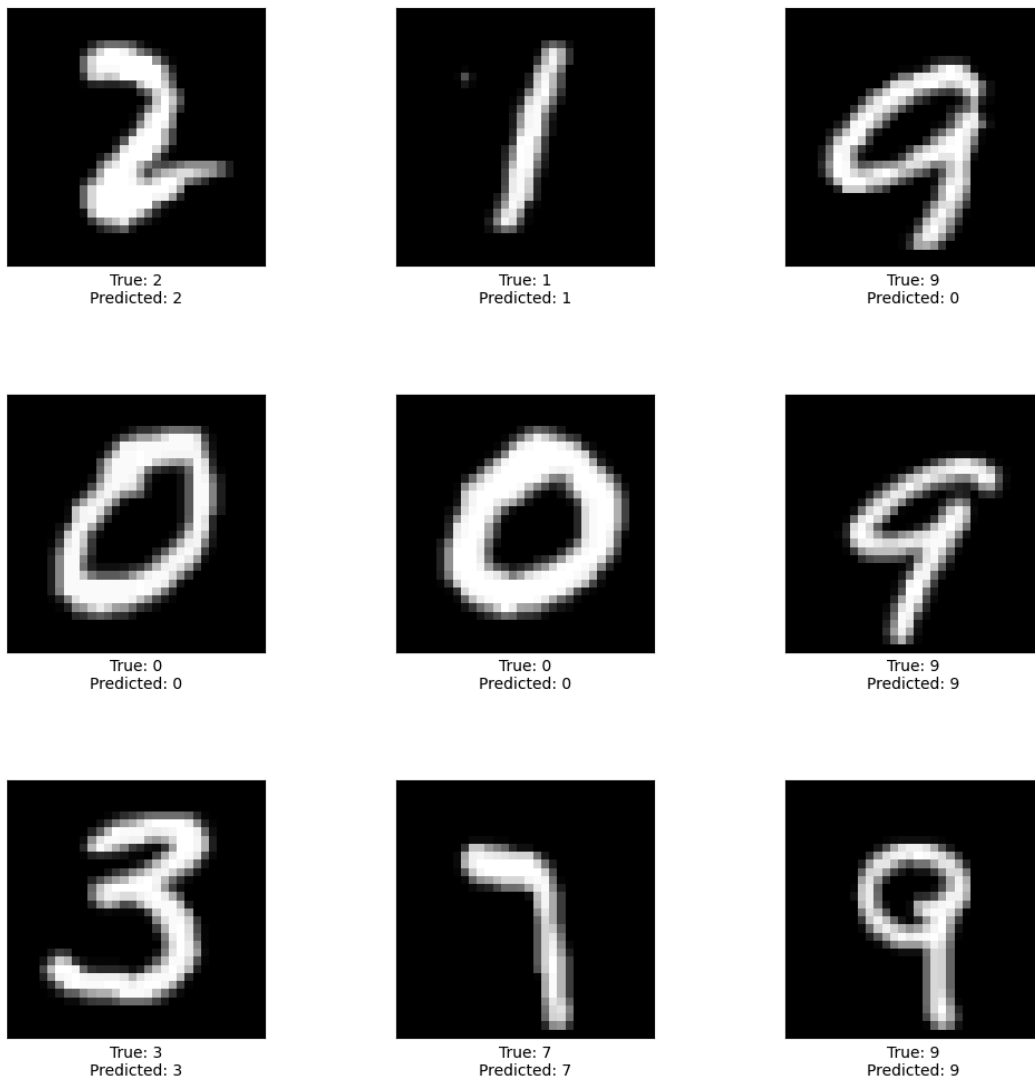
```

    ax.imshow(sample_images[i].reshape(32, 32), cmap='gray')
    ax.set_xlabel(f"True: {sample_labels[i]}\nPredicted: {sample_predictions[i]}")
    ax.set_xticks([])
    ax.set_yticks([])

plt.show()

test_images_gray = np.dot(test_images[:, :, 3], [0.2989, 0.5870, 0.1140])
random_indices = np.random.choice(len(test_images_gray), 9, replace=False)
sample_images = test_images_gray[random_indices]
sample_labels = test_true_classes[random_indices]
sample_predictions = test_predictions_classes[random_indices]
display_sample(sample_images, sample_labels, sample_predictions)

```



The output might change every time you run the code

Let's summarize what we just did - Instead of training a model from zero:

- We took a model that already "knows how to see images"
- We removed its last decision layer
- We replaced it with a new one for MNIST digits
- Then we fine-tuned it slightly

So in simple terms: We didn't teach the model how to see, We only taught it how to recognize digits

Summary

Transfer Learning is a practical and powerful approach in Machine Learning, but it often feels unfamiliar to beginners because the idea seems simple on the surface while being conceptually deeper underneath. Instead of training a model from scratch, we reuse a model that has already learned useful patterns from a large dataset and adapt it to a new but related task.

However, two key challenges usually determine whether Transfer Learning works well:

1. What kind of pre-trained models are usable?

Not every pre-trained model is suitable for every problem. The model must have learned features that are relevant to the new task (for example, image features for image tasks, or language patterns for text tasks).

2. How should the model be adapted?

We must decide which parts of the model to freeze and which parts to fine-tune. Freezing preserves general knowledge, while fine-tuning allows the model to adjust to the new dataset.

In essence, Transfer Learning is about reusing learned knowledge efficiently, rather than starting from zero, while carefully balancing reuse and adaptation to achieve good performance on new tasks.

References

<https://www.geeksforgeeks.org/machine-learning/ml-introduction-to-transfer-learning/>

<https://www.ibm.com/think/topics/transfer-learning>

<https://builtin.com/data-science/transfer-learning>

<https://www.youtube.com/watch?v=YaBX-eNdYGw>

<https://www.youtube.com/watch?v=MQkVIYzpK-Y&sttick=1>

Next Section:

46. Active Learning